

Universidad
Politécnica
de Cartagena

UNIVERSIDAD POLITÉCNICA DE CARTAGENA
ARQUITECTURA DE HARDWARE

MULTIPLICADOR SECUENCIAL

Práctica 1

Autores

Carmen Segura Martinez
André Yermak Naumenko

October 14, 2025

Contents

1	Introducción	1
2	Edición del Código VHDL	3
2.1	1. Sustitución del bus de entrada	3
2.2	2. Eliminación de weQ y weM	5
2.3	3. Control de la salida del resultado	6
3	Simulación TestBench	7

Chapter 1

Introducción

El objetivo de esta práctica es actualizar y reestructurar la implementación del multiplicador secuencial estudiado en clase, desarrollando una versión alternativa del diseño original. Esta nueva versión busca modificar la forma en que se gestionan las señales de entrada y control, adaptando la arquitectura del sistema sin alterar su funcionamiento básico.

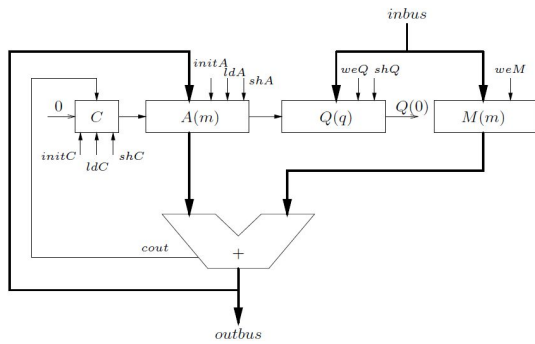
En la Figura 1.1 se muestran, de forma comparativa, la estructura inicial del multiplicador secuencial y la modificación propuesta. A continuación, se indican los cambios que deben incorporarse en el nuevo diseño:

- Sustituir el bus de entrada `inbus` de 8 bits por dos buses: `inbusQ` de 18 bits y `inbusM` de 24 bits, con posibilidad de introducir simultáneamente los datos del Multiplicando y del Multiplicador.
- Eliminar de la entidad los puertos de entrada `weQ` y `weM` y generar las señales de control equivalentes mediante la FSM.
- Impedir que el resultado parcial del multiplicador aparezca a la salida hasta no obtener el resultado final.
- Modificar el testbench de simulación y comprobar el correcto funcionamiento del nuevo diseño.
- Redactar un informe de la práctica que incluya: portada (Título de la Práctica, N° de práctica, autores, etc.), documento con los ficheros VHDL modificados y una breve explicación de las modificaciones realizadas y de los principales resultados de simulación.

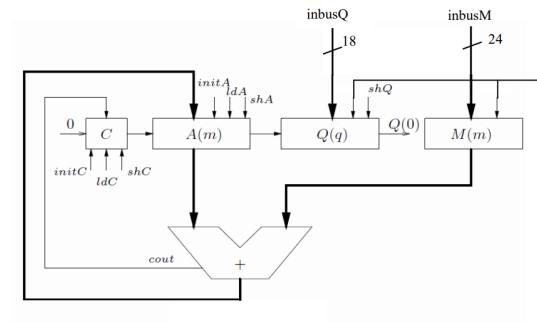
El desarrollo de esta práctica implica adaptar la estructura del multiplicador secuencial para incorporar las nuevas interfaces de entrada y modificar la lógica de control, reemplazando las señales de escritura externas por un control interno gestionado por la FSM. Asimismo, se deberá revisar la secuencia de estados y asegurar que el flujo de datos entre registros sea coherente con las nuevas condiciones de entrada y salida.

Finalmente, será necesario realizar una simulación completa que permita validar el funcionamiento correcto del diseño, verificando que el resultado final se presenta únicamente al completarse el proceso de multiplicación y que no se generan valores intermedios en la salida.

El informe final deberá reflejar la estructura y comportamiento de esta nueva versión, describiendo la implementación realizada, así como las diferencias entre la arquitectura original (Figura 1.1a) y la versión modificada (Figura 1.1b).



(a) Estructura de partida del multiplicador secuencial.



(b) Modificación propuesta a la estructura del multiplicador secuencial.

Figure 1.1. Comparación entre la estructura original (a) y la versión modificada (b) del multiplicador secuencial.

Chapter 2

Edición del Código VHDL

2.1 1. Sustitución del bus de entrada

En primer lugar, declaramos la nueva estructura de la entidad con la nueva longitud de bits correspondientes (véase Código 2.1):

```
1  entity mult_sec is
2      Port ( inbus_Q  : in std_logic_vector(17 downto 0);
3            inbus_M   : in std_logic_vector(23 downto 0);
4            outbus    : out std_logic_vector(41 downto 0);
5            I         : in std_logic;
6            rst       : in std_logic;
7            clk       : in std_logic);
8  end mult_sec;
```

Listing 2.1. Definición de la entidad con los nuevos buses de entrada.

Y lo mismo para nuestras señales internas (Código 2.2):

```
1  architecture Behavioral of mult_sec is
2
3      type type_state is (INICIO, SUMACU, DESDEC);
4      signal state, nextstate: type_state;
5
6      signal CA, pp: std_logic_vector(24 downto 0);
7      signal Q: std_logic_vector(17 downto 0);
8      signal M: std_logic_vector(23 downto 0);
9      signal init, ld, sh: std_logic;
10     signal cnt: unsigned (4 downto 0);
11     signal Z: std_logic;
12     signal enable_outbus: std_logic;
13
14 begin
```

Listing 2.2. Declaración de señales internas de la arquitectura.

A continuación, se actualiza el tamaño del registro CA en el proceso de iniciación (Código 2.3):

```
1      -- Registro C y A:
2      process(rst, clk)
3      begin
4          if (rst='1') then
5              CA <= (others=>'0');
6          elsif rising_edge(clk) then
7              if (init='1') then
8                  CA <= (others=>'0');
9              elsif (ld='1') then
10                 CA <= pp;
11             elsif (sh='1') then
12                 CA <= '0' & CA(24 downto 1);
13             end if;
14         end if;
15     end process;
16
```

Listing 2.3. Proceso del registro C y A con nuevo tamaño.

De la misma forma, se ajustan los registros Q y M para que se adapten al nuevo tamaño de los buses de entrada (Código 2.4):

```
1      -- Registro Q:
2      process(rst, clk)
3      begin
4          if (rst='1') then
5              Q <= (others=>'0');
6          elsif rising_edge(clk) then
7              if (init='1') then
8                  Q <= inbus_Q(17 downto 0);
9              elsif (sh='1') then
10                 Q <= CA(0) & Q(17 downto 1);
11             end if;
12         end if;
13     end process;
14
15     -- Registro M:
16     process(rst, clk)
17     begin
18         if (rst='1') then
19             M <= (others=>'0');
20         elsif rising_edge(clk) then
21             if (init='1') then
22                 M <= inbus_M(23 downto 0);
23             end if;
24         end if;
25     end process;
26
```

Listing 2.4. Procesos de los registros Q y M.

2.2 2. Eliminación de weQ y weM

En este apartado, se eliminan las señales de control weQ y weM de la entidad y se utiliza una única señal de control init para gestionar la carga de los registros Q y M. Su implementación ya puede observarse en el código anterior (véase el Código 2.4).

Por otro lado, la señal init se genera y se gestiona en la FSM, como se muestra a continuación:

```
1  process(state, I, Q(0), Z)
2  begin
3      init <= '0'; ld <= '0'; sh <= '0'; enable_outbus <= '0';
4      nextstate <= state;
5      case state is
6          when INICIO =>
7              enable_outbus <= '1';
8              if (I='1') then
9                  init <= '1';
10                 nextstate <= SUMACU;
11             else
12                 init <= '0';
13                 nextstate <= INICIO;
14             end if;
15         when SUMACU =>
16             enable_outbus <= '0';
17             if (Q(0)='1') then
18                 ld <= '1';
19             else
20                 ld <= '0';
21             end if;
22             nextstate <= DESDEC;
23         when DESDEC =>
24             if (Z='1') then
25                 nextstate <= INICIO;
26             else
27                 nextstate <= SUMACU;
28             end if;
29             sh <= '1';
30         when others =>
31             nextstate <= INICIO;
32     end case;
33 end process;
```

Listing 2.5. Generación de la señal init en la FSM.

Con esta implementación, ya no es necesario de usar dos ciclos de reloj para cargar los registros Q y M, sino que ambos se cargan simultáneamente en un único ciclo cuando la señal init se activa.

2.3 3. Control de la salida del resultado

La forma en la que se ha controlado la salida del resultado es mediante la señal `enable_outbus`, que se activa únicamente en el estado `INICIO` de la FSM (véase Código 2.5). Una forma sencilla que, llegado al estado `INICIO`, significa que la operación ha finalizado, por lo tanto no requiere nada más que una verificación al cargar `outbus`:

```
1      --Salida:  
2      outbus <= (CA(23 downto 0) & Q) when (enable_outbus= '1') else  
3      (others => '0');
```

Listing 2.6. Carga del bus de salida.

Chapter 3

Simulación TestBench

Una vez realizado el rediseño del multiplicador secuencial, se procede a actualizar también el `testbench` para adaptarlo a las nuevas características y restricciones del sistema. La declaración del componente y sus puertos, las señales internas y la instancia del DUT se muestran en el Código 3.1.

Tras la declaración, asignamos valores a las señales para verificar si el diseño funciona correctamente. En primer lugar, se genera un producto de $13 \cdot 200$ (resultado = 2600) y luego otro arbitrario de $123.277 \cdot 9.381.064$ (resultado = 1.156.469.426.728) mostrado en el código 3.2.

Podemos verificar el correcto funcionamiento del diseño observando las señales en la simulación (Figura 3.1).

```

1  ARCHITECTURE behavior OF testbench_mult_sec IS
2
3      COMPONENT mult_sec
4      PORT(
5          inbus_Q   : IN std_logic_vector(17 downto 0);
6          inbus_M   : IN std_logic_vector(23 downto 0);
7          I         : IN std_logic;
8          rst       : IN std_logic;
9          clk       : IN std_logic;
10         outbus    : OUT std_logic_vector(41 downto 0)
11     );
12     END COMPONENT;
13
14     SIGNAL inbus_Q : std_logic_vector(17 downto 0) := (others=>'0');
15     SIGNAL inbus_M : std_logic_vector(23 downto 0) := (others=>'0');
16     SIGNAL outbus  : std_logic_vector(41 downto 0);
17     SIGNAL I       : std_logic := '0';
18     SIGNAL rst     : std_logic := '1';
19     SIGNAL clk     : std_logic := '0';
20
21     constant periodo: time := 100 ns;
22
23 BEGIN
24
25     uut: mult_sec
26     PORT MAP(
27         inbus_Q => inbus_Q,
28         inbus_M => inbus_M,
29         outbus  => outbus,
30         I       => I,
31         rst     => rst,
32         clk     => clk
33     );
34

```

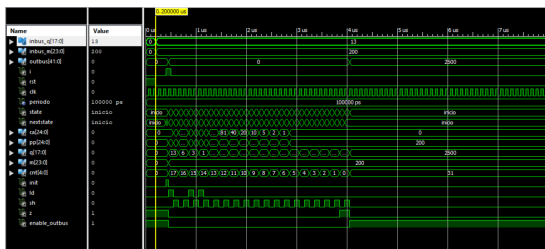
Listing 3.1. Generación de la señal init en la FSM.

```

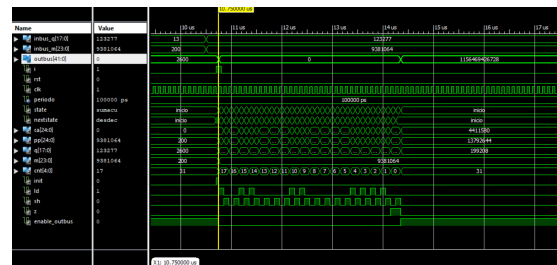
1      -- Estímulos
2      tb : PROCESS
3      BEGIN
4          rst <= '1';
5          wait for 2*periodo;
6          rst <= '0';
7
8          -- Carga de operandos
9          inbus_M <= "000000000000000011001000"; -- M = 200
10         inbus_Q <= "000000000000001101";      -- Q = 13
11         wait for 2*periodo;
12
13         I <= '1'; -- inicio de la multiplicación
14         wait for periodo;
15         I <= '0';
16
17         -- Espera suficiente tiempo para completar el proceso
18         wait for 100*periodo;
19
20         -- Carga de operandos
21         inbus_M <= "100011110010010011001000"; -- M = 9381064
22         inbus_Q <= "011110000110001101";      -- Q = 123277
23         wait for 2*periodo;
24
25         I <= '1'; -- inicio de la multiplicación
26         wait for periodo;
27         I <= '0';
28
29         -- Espera suficiente tiempo para completar el proceso
30         wait for 100*periodo;
31         wait;
32     END PROCESS;
33

```

Listing 3.2. Generación de la señal init en la FSM.



(a) Primer producto realizado



(b) Segundo producto realizado

Figure 3.1. Simulación del testbench mostrando los dos productos realizados.